Programa:	AccessImage 5.20	
Protección:	Limitado a 50 ejecuciones + Nagscreen	
Objetivos:	Evitar la restricción.	
URL descarga:	---	
Dificultad:	No es nada difícil	
Herramientas:	OllyDbg y w32dasm	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 19-07-2005

Bueno ya sabemos: Limitación de 50 ejecuciones + una nagscreen, cargamos el programa en OllyDbg y empezamos a tracear a ver qué vemos, como primer apunte decir que estaba ejecutando el programa normalmente abri el Olly lo volví alanzar y salió el mensaje de Expired, no sé si esto fue la caoualidad o me detectó el Olly, aún no lo sé pero da igual ya porque está expirado ;(por todo el morro, vaya modales!!

Vamos a la caza de la Nag ya que ha surgido así la caso no lo hemos buscado, y es así de feo, jeje..

Esta ya no nos dejará pasar más, iremos traceando hasta ver donde sale cuando salga buscaremos algún salto que la evite, jeje.. un método que a veces falla pero no es el caso ya que llegamos aquí:

```
:0042FA93   E8 F80A0000      CALL AccessIm.00430590              ;mira si nos quedan ejecuciones..!!
:0042FA98   A3 88C24B00      MOV DWORD PTR DS:[4BC288],EAX       ;eax hacede contador y guarda el resultado
:0042FA9D   83F8 00          CMP EAX,0
:0042FAA0   0F8C A9040000    JL AccessIm.0042FF4F                ;si pasamos de las 50 ejecuciones salimos
```

el salto 'jl 42ff4f' nos envía de cabeza al cartel, dentro del call accede al registro a leer el valor que le informa el número de ejecuciones que hemos gastado, el contador lo tiene en:

HK_USERS\DEFAULT\Sofware\Acccg //crt1

El contenido del valor crt1 está digamos que encriptado para que no se pueda leer directamente su contenido, si entramos en el call anterior vemos que efectivamente accede a este valor y lo lee, hace sus operaciones y al final determina lo que tiene que hacer si mostrar el cartel de Expired o sacar el programa

```
:00430B92   E8 AA790600      CALL <JMP.&ADVAPI32.RegCloseKey>      ; \RegCloseKey
:00430B97   83FB 32          CMP EBX,32          ;ebx tiene el número de ejecuciones que hemos gastado..!!
:00430B9A   0F82 F8000000    JB AccessIm.00430C98  ;si es inferios a las 50 permitidas salta…
```

Esta zona es algo más grande, veamos un fragmento:

```
:00430B22  C70424 84BF4B>    MOV DWORD PTR SS:[ESP],AccessIm.004BBF84 ;|"C:\WINDOWS\System32\xqpdai.dll"
:00430B29  E8 61740600       CALL <JMP.&KERNEL32.BeginUpdateResourceA>; \BeginUpdateResourceA
:00430B2E  8BF0              MOV ESI,EAX
:00430B30  85F6              TEST ESI,ESI
:00430B32  0F84 68000000     JE AccessIm.00430BA0
:00430B38  83C4 E8           ADD ESP,-18
```

```
:00430B3B  C74424 14 180>      MOV DWORD PTR SS:[ESP+14],18
:00430B43  C74424 10 6CB>      MOV DWORD PTR SS:[ESP+10],AccessIm.004BB>
:00430B4B  C74424 0C 000>      MOV DWORD PTR SS:[ESP+C],0
:00430B53  C74424 08 466>      MOV DWORD PTR SS:[ESP+8],6E46
:00430B5B  C74424 04 0A0>      MOV DWORD PTR SS:[ESP+4],0A
:00430B63  893424              MOV DWORD PTR SS:[ESP],ESI
:00430B66  E8 4E740600         CALL <JMP.&KERNEL32.UpdateResourceA>     ; \UpdateResourceA
:00430B6B  85C0                TEST EAX,EAX
:00430B6D 74 31                JE SHORT AccessIm.00430BA0
:00430B6F  83C4 F8             ADD ESP,-8
:00430B72  C74424 04 000>      MOV DWORD PTR SS:[ESP+4],0
:00430B7A 893424               MOV DWORD PTR SS:[ESP],ESI
:00430B7D E8 1F740600          CALL <JMP.&KERNEL32.EndUpdateResourceA>  ; \EndUpdateResourceA
:00430B82  85C0                TEST EAX,EAX
:00430B84  74 1A               JE SHORT AccessIm.00430BA0
:00430B86  83C4 FC             ADD ESP,-4
:00430B89  8B85 98FEFFFF       MOV EAX,DWORD PTR SS:[EBP-168]
:00430B8F  890424              MOV DWORD PTR SS:[ESP],EAX
:00430B92  E8 AA790600         CALL <JMP.&ADVAPI32.RegCloseKey>         ; \RegCloseKey
:00430B97 83FB 32              CMP EBX,32
:00430B9A 0F82 F8000000        JB AccessIm.00430C98
```

Parece que accede a la ruta “C:\Windows\System32\” y busque el archivo “xqpdai.dll” con una llamada a “BeginUpdateResourceA” después le sigue otra a “UpdateResourceA” y finalmente otra más a “EndUpdateResourceA” si el retorno de esta última llamada es eax=0 vemos la nag de Expired sin verificar el contador ‘crt1’ algo muy extraño pero es así, no sé con que intención estará progamado ni lo que estrará verificando pero bueno, basta con nopear siempre en :430b84 y saltar en : 430b9a para que no expire nunca pero seguiremos viendo la nag inicial, así que entraremos más a fondo en el tema, un apunte: retomamos el codigo:

```
:0042FA93   E8 F80A0000        CALL AccessIm.00430590                 ;mira si nos quedan ejecuciones..!!
:0042FA98   A3 88C24B00        MOV DWORD PTR DS:[4BC288],EAX          ;eax hacede contador y guarda el resultado
:0042FA9D   83F8 00            CMP EAX,0
:0042FAA0   0F8C A9040000      JL AccessIm.0042FF4F                   ;si pasamos de las 50 ejecuciones salimos
```

aquí recoradmos que es donde mira si estamos en modo Expired o por el contrario nos quedan ejecuciones, bien pues vemos que eax guarda su valor en [4BC288], si ponemos un “BP on Memory Acces” pararemos en varios lugares, si cada vez que pare y le pase el valor a otra celda de memoria hacemos lo mismo llegaremos a esto:

```
:0017EF11    A1 487B1800       MOV EAX,DWORD PTR DS:[187B48]
:0017EF16    81F8 C8000000     CMP EAX,0C8
:0017EF1C    0F85 B0000000     JNZ access_1.0017EFD2
:0017EF22    83C4 F8           ADD ESP,-8
:0017EF25    A1 18B21800       MOV EAX,DWORD PTR DS:[18B218]
:0017EF2A    894424 04         MOV DWORD PTR SS:[ESP+4],EAX
:0017EF2E    891C24            MOV DWORD PTR SS:[ESP],EBX
:0017EF31    E8 4E760000       CALL <JMP.&GDI32.SetTextColor>
:0017EF36    83C4 F8           ADD ESP,-8
:0017EF39    C70424 147C1800   MOV DWORD PTR SS:[ESP],access_1.00187C14 ; "Thank you for purchasing…"
```

Veamos que los saltos delatan que estamos en el archivo “acces_1_lo_que_sea” pero si miramos en el directorio de instalacion no vemos ningun archivo con este nombre, pues se trata de “accessimg.dll” lo que ocurre es que no puede usar el nombre original porque ya esta el ejecutable con extension .exe con este mismo nombre por eso le añade la coletilla _1, bien abrimos “accessimg.dll” con w32dasm y vemos que las direcciones empiezan a partir de :401000 y en Olly veiamos direcciones tipo :17EF11 esto es porque al ser una dll la carga dinamicamente y pueden cambiar las direcciones de esta manera pero no pasa nada, fijemonos en la instrucción:

```
:0017EF16    81F8 C8000000     CMP EAX,0C8
```

es muy poco comu asi que si lanzamos una busqueda llegamos rapido al sitio:

```
:0040EF11 A1487B4100           mov eax, dword ptr [00417B48]
:0040EF16 81F8C8000000         cmp eax, 000000C8
:0040EF1C 0F85B0000000         jne 0040EFD2
:0040EF22 83C4F8               add esp, FFFFFFF8
:0040EF25 A118B24100           mov eax, dword ptr [0041B218]
:0040EF2A 89442404             mov dword ptr [esp+04], eax
```

```
:0040EF2E 891C24                mov dword ptr [esp], ebx

* Reference To: GDI32.SetTextColor, Ord:0000h
                |
:0040EF31 E84E760000            Call 00416584
:0040EF36 83C4F8                add esp, FFFFFFF8

* Possible StringData Ref from Data Obj ->"Thank you for purchasing Access "
                                        ->"Image 5.20, graphics editor software."
                |
:0040EF39 C70424147C4100        mov dword ptr [esp], 00417C14
```

Parece un buen lugar no? que pasaria si hubiera un “0C8” en [00417B48] podriamos comprobarlo, como sabemos que el valor se calcula en el propio ejecutable podemos parchear alli mismo en el call que calcula el numero de ejecuciones transcurridas, una salida facil es esta:

```
:00430B97   83FB 32             CMP EBX,32
:00430B9A   0F82 F8000000       JB AccessIm.00430C98
:00430BA0   83C4 FC             ADD ESP,-4
:00430BA3   8B85 98FEFFFF       MOV EAX,DWORD PTR SS:[EBP-168]
:00430BA9   890424              MOV DWORD PTR SS:[ESP],EAX
:00430BAC.  E8 90790600         CALL <JMP.&ADVAPI32.RegCloseKey>        ; \RegCloseKey
:00430BB1   B8 FFFFFFFF         MOV EAX,-1
```

don de podemos cambiar dos cosas para asegurarnos salir con eax=0C8h, serian:

```
00430BB1     B8 FFFFFFFF        MOV EAX,-1              POR:
00430BB1     B8C8000000         MOV EAX, 0C8

:00430B9A   0F82 F8000000       JB AccessIm.00430C98    POR:
:00430B9A   909090909090        NOPS
```

El segundo cambio por si acaso quiere saltar y desviarse de esta salida ya que el call tiene mas lugares de retorno aparte de este, asi saldra por aquí y con eax=0C8, jeje..

Probamos los cambios y entramos en modo Registered ;DDD

Y este es el parche, ahora en la ventana de Registro ya no dice más que es Trial, el único pero es que no pone nuestro nombre pero bueno que más da, el trabajo está hecho igual, se puede mejorar claro pero no está mal.

Si alguien tiene una duda, sugerencia, crítica o lo que sea me lo puede hacer saber escribiéndome a:

syxe00@yahoo.es
syxe00@hotmail.com

Contestaré a todas los envíos para lo que sean.